Predicting the Acceptance of Pull Requests Opened by AI Coding Agents: A Data Mining Study on the AIDev Dataset

Musa AL AHMED
CME519 Data Mining – Midterm Project Report
Graduate School, Istanbul Topkapı University
Istanbul, Türkiye
Student No: 26221001014

***Abstract*—Autonomous AI coding agents such as OpenAI Codex, Devin, GitHub Copilot, Cursor and Claude Code now open pull requests (PRs) in real open-source projects, but a large share of these PRs is never merged, which wastes maintainer effort. This paper studies whether the outcome of an agent-authored PR can be predicted at the moment it is opened. Using the curated subset of the public AIDev dataset, we integrate five relational tables into one PR-level dataset of 332,406 closed PRs from 22,357 repositories (25.8% rejected) produced by 6 agents. Following CRISP-DM, we engineer date, text, author and code-change features, build a leakage-safe preprocessing pipeline, compare eight feature-selection methods and evaluate a baseline and four classifiers (KNN, Logistic Regression, Decision Tree, Random Forest) with 5-fold stratified cross-validation. Because rejected PRs are the minority class, models are compared mainly by the precision–recall AUC (PR-AUC) of the rejected class. The best model, Random Forest, reaches a cross-validated PR-AUC of 0.674 versus 0.258 for a random baseline, and a ROC-AUC of 0.815 on the held-out test set. Author and repository characteristics (author followers and account age, repository forks and stars) are the strongest predictors, followed by the shape of the code change.**

***Index Terms*—data mining, pull request acceptance, AI coding agents, imbalanced classification, feature selection, CRISP-DM**

I. Introduction

Pull-based development is the dominant contribution model on GitHub: a contributor proposes a change as a pull request (PR) and a project maintainer decides whether to merge it [1]. Since 2025, a new type of contributor has appeared: autonomous coding agents that write code and open PRs on behalf of developers, a shift described as "Software Engineering 3.0" [5]. Agents can produce PRs much faster than humans, but every PR still needs human review, and rejected PRs consume reviewer time without adding value.

Problem. Given only the information available when an agent-authored PR is opened, predict whether it will be merged or closed without merge. An accurate model can prioritise review effort and give early feedback to developers who use agents.

Contributions. (i) We build a leakage-aware, PR-level dataset of 332,406 closed agent PRs by integrating five AIDev tables, aggregating about 12.1 million file-level change records; (ii) we engineer 18 features from dates, text, authors and code changes; (iii) we compare four outlier treatments, three scalers and eight feature-selection methods; and (iv) we compare four classifiers with 5-fold stratified cross-validation, evaluate the best one on a held-out test set and explain it with permutation importance and a leakage ablation.

The rest of the paper is organised as follows: Section II reviews related work, Section III describes the data and the exploratory analysis, Section IV the method, Section V the results, and Section VI discusses limitations and future work.

II. Related Work

PR acceptance in human development. Gousios *et al.* [1] analysed pull-based development on GitHub and showed that whether a PR touches recently modified code, the project's size and the contributor's track record influence the merge decision. Tsay *et al.* [3] found that both social factors (e.g., the submitter's prior interaction and followers) and technical factors (e.g., size and tests) affect acceptance. Zhang *et al.* [4] systematically re-examined the factors proposed in earlier studies on a large dataset and confirmed that contributor experience, PR size and project characteristics are among the most consistent predictors. Gousios *et al.* [2] described the integrator's perspective, where code quality and conformance to project norms dominate the decision.

AI coding agents on GitHub. Li *et al.* introduced the AIDev dataset of agent-authored PRs and reported that agent PRs are produced at high speed but are accepted less often than human PRs [5], [6]. Watanabe *et al.* [7] studied PRs created with Claude Code and found that most were merged but many needed human revision. These works describe acceptance rates; this paper instead builds and evaluates predictive models for the merge outcome of individual agent PRs.

Methodological background. We follow the CRISP-DM process [8] and standard techniques from [9]. Because rejected PRs are the minority class, we report precision–recall curves, which are more informative than ROC curves on imbalanced data [10], and we take explicit care to avoid data leakage [11]. All experiments use scikit-learn [12].

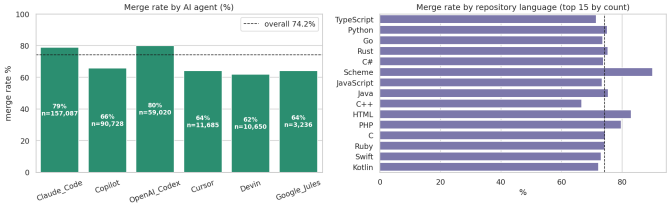


Fig. 1. Merge rate by AI agent and by repository language (closed PRs). The dashed line is the overall merge rate.

III. Data and Exploratory Analysis

A. Data Collection and Integration

AIDev is published on Hugging Face (`hao-li/AIDev`). We use its curated subset of repositories with more than 100 GitHub stars and five tables: `pull_request` (361,296 PRs), `repository` (25,402 repositories), `all_user` (author profiles), `pr_commits` and `pr_commit_details` (12,116,310 file-level change records). The file-level table (12 GB) was streamed in batches of one million rows and aggregated to one row per PR. Only closed PRs have a known outcome, which leaves 332,406 PRs; the target is *merged*=1 if the PR has a merge timestamp and 0 otherwise. The data are raw: they contain missing values (19 columns, up to 27.3% missing), heavy-tailed distributions and class imbalance (74.2% merged vs. 25.8% rejected).

Tables that describe events after the PR is opened (reviews, comments, timeline) were deliberately not used as features, because they would reveal the outcome. Fig. 2 illustrates this: time-to-close differs strongly between merged and rejected PRs and would be a leaking feature.

B. Exploratory Findings

All 17 numeric attributes are strongly right-skewed (e.g., lines added, repository stars, author followers), so medians were preferred over means and log transforms were tested. The IQR rule flags many more points than the $|z| > 3$ rule because of the skew; these extreme values are real (very large PRs, very popular repositories), so they were treated rather than removed. The merge rate differs clearly between agents, from 61.9% to 79.9% (Fig. 1), and the strongest single numeric correlate of the target is `user_followers` (Spearman $\rho = 0.233$). 4 pairs of change-size features are almost redundant ($|\rho| > 0.85$), which motivates feature selection.

IV. Methodology

A. Feature Engineering

Features were derived row by row (no information from other rows), which is safe before the train/test split: *date* features from the creation time (hour, weekday, weekend flag, month); *author* account age at PR creation; *text* features (title length and word count, description length, empty-description flag, conventional-commit title type such as *feat*/*fix*/*docs*, checklist and test mentions); *code-change* ratios (net lines, lines per file, share of test, documentation and configuration

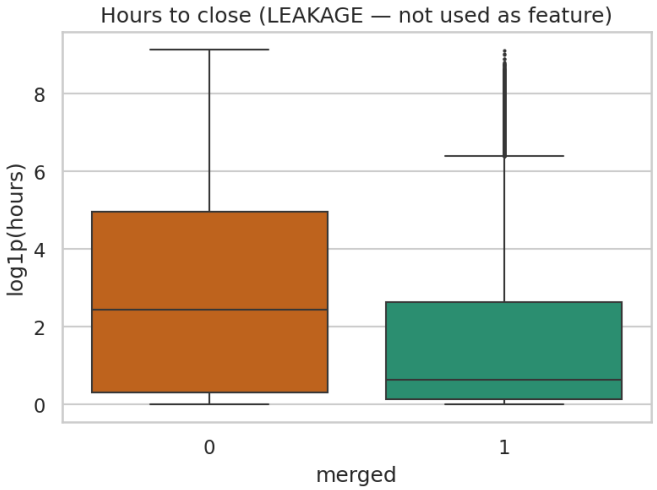


Fig. 2. Hours to close by class. This post-creation information is a leakage feature and is excluded from all models.

files); and a repository licence flag. The raw dates and free text were then dropped. The final model input has 38 features: 34 numeric and 4 categorical (agent, repository language, fork flag, title type).

B. Leakage-Safe Preprocessing

The data were split first into 80% training and 20% test sets, stratified by the target. Every step that learns from data is wrapped in a scikit-learn `Pipeline` and fitted on training folds only: median imputation with missing-value indicators for numeric features and a constant category for categorical ones; outlier treatment; one-hot encoding in which categories below 1% frequency are merged into an "infrequent" column (the language attribute has 184 values); and z-score standardisation, which distance-based and linear models require.

C. Feature Selection

Eight feature sets were compared: all features; three filters (variance threshold, correlation filter $|r| > 0.9$, top-15 mutual information); two wrappers (recursive feature elimination and forward selection with Logistic Regression); and two embedded methods (L1-regularised Logistic Regression and Random-Forest importance). To prevent leakage and keep runtime feasible, selectors were fitted on a 15,000-PR selection sample and evaluated on a disjoint 40,000-PR sample of the training set.

D. Models and Evaluation Protocol

We compare a stratified random baseline, K-Nearest Neighbours (K chosen by grid search with 5-fold CV), Logistic Regression, a Decision Tree and a Random Forest; the latter three use balanced class weights. KNN stores all training points and computes distances at prediction time, so it was trained on a 60,000-PR stratified sample. All models are evaluated with the same 5-fold stratified cross-validation. Rejected PRs are treated as the positive class for PR-AUC, recall and precision;

TABLE I
FEATURE-SELECTION COMPARISON: PR-AUC OF THE REJECTED CLASS (5-FOLD CV). F = FILTER, W = WRAPPER, E = EMBEDDED.

Method	#cols	LogReg	KNN	RF	Mean
E: L1 LogReg	64	0.576	0.548	0.630	0.585
F: variance threshold	84	0.577	0.543	0.625	0.582
All features	86	0.577	0.543	0.624	0.581
F: correlation $\|r\|>0.90$	63	0.577	0.538	0.623	0.579
E: RandomForest importance	43	0.568	0.543	0.625	0.578
W: forward selection (10)	10	0.557	0.536	0.602	0.565
F: mutual information (top 15)	15	0.546	0.528	0.597	0.557
W: RFE-LogReg (15)	15	0.544	0.527	0.584	0.552

TABLE II
5-FOLD STRATIFIED CV ON THE TRAINING SET (REJECTED = POSITIVE CLASS).

Model	PR-AUC	ROC-AUC	F1-mac.	Rec.	Prec.
Random Forest	0.674±0.002	0.816	0.719	0.631	0.559
KNN (K=31, distance)	0.595±0.010	0.762	0.636	0.269	0.772
Decision Tree	0.594±0.003	0.764	0.659	0.657	0.451
Logistic Regression	0.577±0.003	0.753	0.649	0.659	0.437
Baseline (stratified)	0.258±0.001	0.500	0.500	0.256	0.258

ROC-AUC and macro-F1 are also reported. The best model is refitted on the full training set and evaluated once on the test set.

V. RESULTS

A. Preprocessing and Feature Selection

For Logistic Regression the best outlier treatment was a signed log1p transform (PR-AUC 0.577). Standardisation changed KNN's PR-AUC from 0.548 (unscaled) to 0.542. On this data the difference is small, because the log transform already brings most features to comparable ranges; standardisation is kept so that no feature dominates the distance computation.

Table I compares the feature-selection methods. The best average PR-AUC was obtained with *Embedded: L1 LogReg* (64 columns), compared with 0.581 when all 86 columns are used. Methods with far fewer columns reach similar scores, which shows that many encoded columns are redundant.

B. Model Comparison

Table II and Fig. 3 summarise the 5-fold cross-validation. All models outperform the random baseline (PR-AUC 0.258, equal to the rejected share). Random Forest obtains the highest PR-AUC (0.674) with a standard deviation across folds of 0.002, indicating a stable estimate.

C. Test-Set Evaluation

On the untouched test set (66,482 PRs) the Random Forest model achieves the scores in Table III. The confusion matrix and the ROC and precision–recall curves are shown in Fig. 4 and Fig. 5. The test PR-AUC (0.674) is close to the cross-validated value, so the model generalises to unseen PRs.

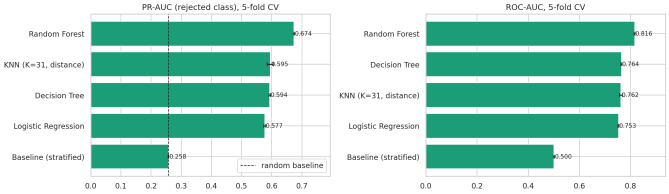


Fig. 3. Cross-validated PR-AUC (rejected class) and ROC-AUC with standard deviation across folds.

TABLE III
TEST-SET PERFORMANCE OF THE SELECTED MODEL.

Metric	Value
PR-AUC (rejected)	0.674
ROC-AUC	0.815
F1 macro	0.718
Balanced accuracy	0.730
Accuracy	0.774
Recall (rejected)	0.638
Precision (rejected)	0.553

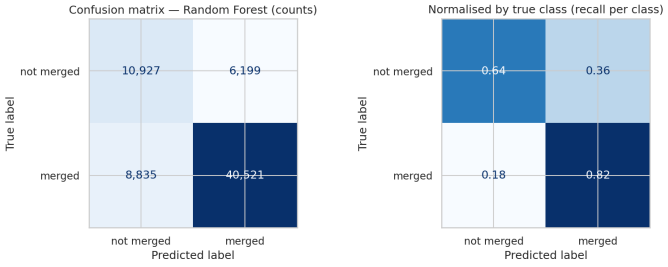


Fig. 4. Confusion matrix on the test set (counts and recall per class).

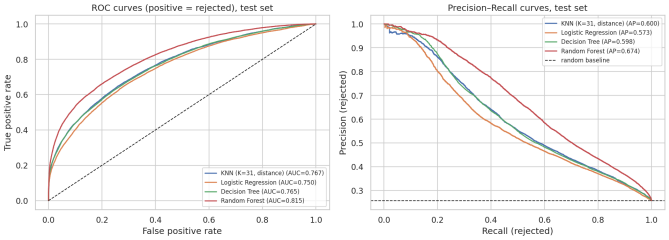


Fig. 5. ROC and precision–recall curves on the test set; the positive class is "rejected".

D. Feature Importance and Leakage Check

Permutation importance (Fig. 6) ranks `user_followers`, `repo_forks`, `repo_stars`, `lines_per_file`, `user_account_age_days` as the most influential features. This agrees with the human-PR literature, where contributor and project characteristics are consistent predictors [3], [4]. The agent identity ranks 8th of 38 features: once author and repository are known, the agent adds comparatively little, although merge rates differ between agents in the EDA.

Commit statistics may include commits pushed after reviewer feedback. Removing all commit-based features changes the cross-validated PR-AUC by -0.065 (to 0.609, still far above

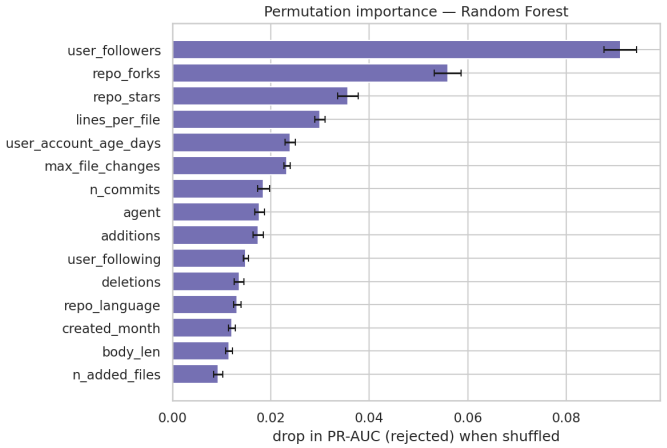


Fig. 6. Permutation importance of the selected model (drop in PR-AUC on the test set when a feature is shuffled).

the 0.258 baseline), so part of the signal comes from commit statistics; restricting them to commits made before the first review is planned for the final project.

VI. Discussion and Conclusion

Findings. The merge outcome of agent PRs can be predicted above chance using only information available at creation time, but the task is hard: the classes overlap strongly in single features, and the best PR-AUC remains moderate. Author reputation and repository popularity matter most, followed by how the change is distributed over files.

Limitations. (i) The random split mixes time periods and repositories, so the model may partly learn repository-specific habits; a time-based or repository-grouped split is planned. (ii) Author follower counts are snapshots taken at data collection, not at PR creation. (iii) Hyperparameters and the decision threshold were not tuned in this midterm phase. (iv) Text was represented only by simple statistics.

Future work (final project). We will tune hyperparameters and the decision threshold, add gradient-boosting models and cost-sensitive learning, restrict commit features to commits made before the first review using the timeline table, and add code-quality features (function length, complexity, nesting, duplication, empty catch blocks, hard-coded secrets) computed with our Clean-Code analysis tool *CleanLens*, into which the final model will be integrated for early detection of problematic AI-generated changes.

Code availability. All notebooks, figures and result tables are available at https://github.com/MUSAALAHMED4/CME519-AIDev-PR-Acceptance.

References

[1] G. Gousios, M. Pinzger, and A. van Deursen, "An exploratory study of the pull-based software development model," in *Proc. 36th Int. Conf. Software Engineering (ICSE)*, 2014, pp. 345–355.

[2] G. Gousios, A. Zaidman, M.-A. Storey, and A. van Deursen, "Work practices and challenges in pull-based development: The integrator's perspective," in *Proc. 37th Int. Conf. Software Engineering (ICSE)*, 2015, pp. 358–368.

[3] J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of social and technical factors for evaluating contribution in GitHub," in *Proc. 36th Int. Conf. Software Engineering (ICSE)*, 2014, pp. 356–366.

[4] X. Zhang, Y. Yu, G. Gousios, and A. Rastogi, "Pull request decisions explained: An empirical overview," *IEEE Trans. Software Engineering*, vol. 49, no. 2, pp. 849–871, 2023.

[5] H. Li, H. Zhang, and A. E. Hassan, "The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering," arXiv:2507.15003, 2025.

[6] H. Li, H. Zhang, and A. E. Hassan, "AIDev: Studying AI coding agents on GitHub," arXiv:2602.09185, 2026.

[7] M. Watanabe, H. Li, Y. Kashiwa, B. Reid, H. Iida, and A. E. Hassan, "On the use of agentic coding: An empirical study of pull requests on GitHub," arXiv:2509.14745, 2025.

[8] P. Chapman *et al.*, "CRISP-DM 1.0: Step-by-step data mining guide," SPSS Inc., 2000.

[9] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann, 2011.

[10] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.

[11] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[12] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.